Dynamic Programming

Compiled by Thapanapong Rukkanchanunt

Origin of Recursion

- ตัวอย่างแรก ๆ ของการใช้แนวคิดแบบ Recursion ปรากฏขึ้นในอินเดียเมื่อกว่า 2,000 ปีก่อน ผ่านการศึกษาจังหวะในบทกวีสันสกฤต
- ในการประพันธ์บทกวีสันสกฤต จะมีการแยกประเภทพยางค์ออกเป็น 2 แบบคือพยางค์เบา (•) ซึ่งมี 1 จังหวะและพยางค์หนัก (—) ซึ่งมี 2 จังหวะ โดยในหนึ่งบรรทัดของบทกวีจะมีจำนวนจังหวะที่กำหนดไว้ตายตัว
- นักวิชาการชื่อ**ปิงคละ**สังเกตว่าจำนวนรูปแบบฉันทลักษณ์ที่มี 4 จังหวะมีด้วยกัน 5 แบบคือ — —, — • •, • — •, • • — และ • • • •

Origin of Recursion

- แม้ว่าปิงคละจะบอกใบ้ในตำราว่านับจำนวนรูปแบบจากขนาดจังหวะที่กำหนดให้ได้อย่างไร แต่กฎการนับแบบชัดเจนต้องใช้เวลาถึงหลายสหัสวรรษจึงจะปรากฏ
- ในคริสต์ศตวรรษที่ 6 นักวิชาการชื่อ**วิรหางกะ**ได้เขียนคำอธิบายตำราของปิงคละ และระบุความสัมพันธ์แบบเรียกซ้ำ (Recurrence) ไว้อย่างชัดเจนว่า:
  "จำนวนรูปแบบฉันทลักษณ์ที่มี n จังหวะ คือผลรวมของจำนวนรูปแบบที่มี n-2 จังหวะ และ n-1 จังหวะ"
  กล่าวคือ $M(n) = M(n-2) + M(n-1)$
- ใช่แล้วครับ มันคือลำดับฟีโบนักชีนั่นเอง (แตกต่างกันตรง Base Case)

Recursive Algorithm

- จากสมการ Recursive ทำให้เราได้อัลกอริทึม Recursive แบบทันที

```
RecFibo(n):
  if n = 0
    return 0
  else if n = 1
    return 1
  else
    return RecFibo(n-1) + RecFibo(n-2)
```

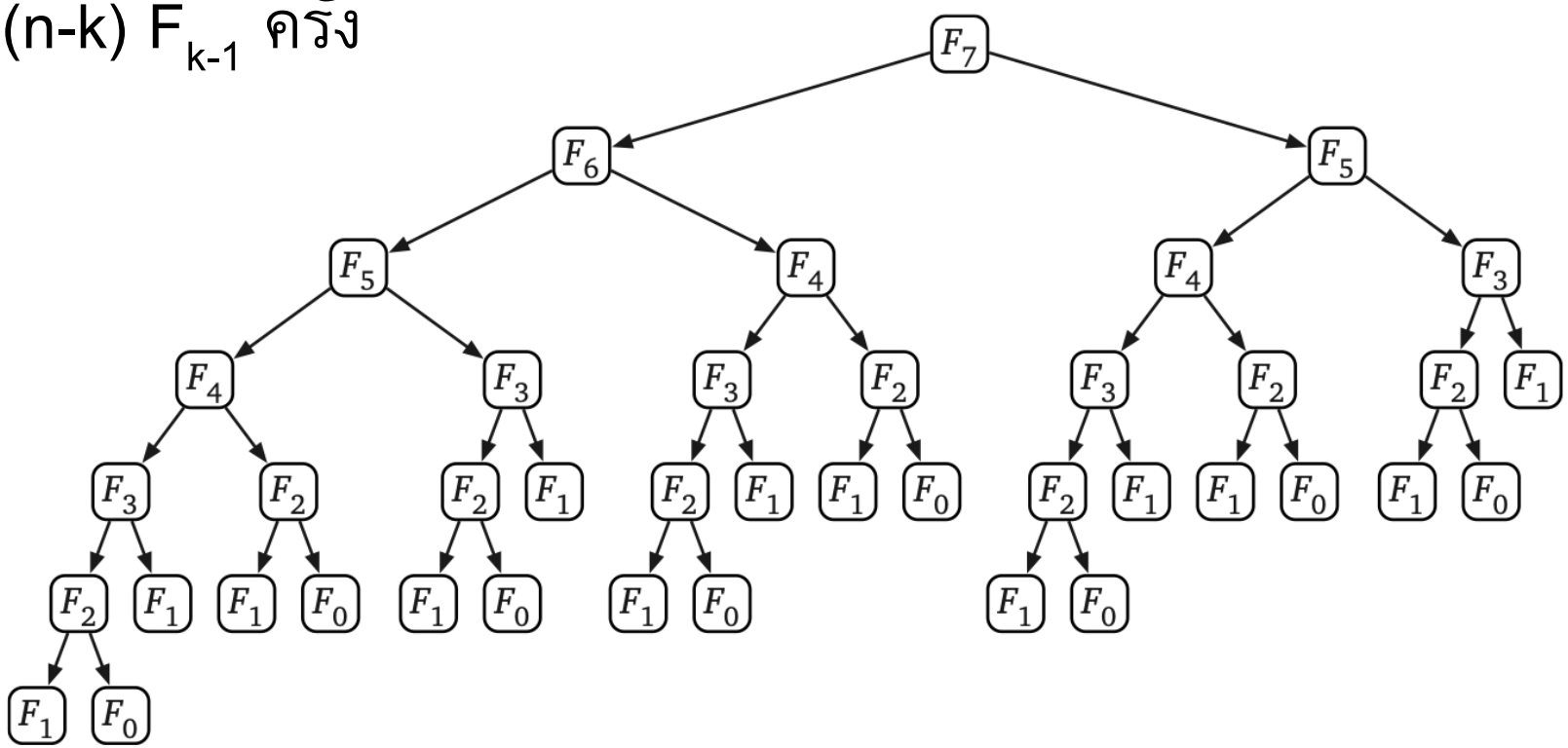
- แน่นอนว่าอัลกอริทึมนี้ช้ามาก (ทุกคนรู้อยู่แล้ว)

Number of Recursive Calls

- ถ้าเราให้ T(n) คือจำนวนครั้งที่เรียกฟังก์ชัน RecFibo ฟังก์ชันนี้มีความสัมพันธ์เวียนเกิดคือ $T(n) = T(n-1) + T(n-2) + 1$ เมื่อ $T(0) = 0$ และ $T(1) = 1$
- ความสัมพันธ์นี้มีสมการสำเร็จรูปคือ $T(n) = 2F_{n+1} - 1$ (พิสูจน์ด้วย Induction)
- นั่นหมายความว่าอัลกอริทึมนี้ใช้เวลาในการคำนวณเป็นสองเท่าของการนับ 1 ถึง F_n เลยทีเดียว

Redundancy

- เหตุผลที่ RecFibo ช้าคือการคำนวณฟิโบนักชีลำดับเดิมซ้ำๆ กัน
- RecFibo(n) เรียก RecFibo(n-1) หนึ่งครั้ง RecFibo(n-2) สองครั้ง … RecFibo(n-k) F_{k-1} ครั้ง

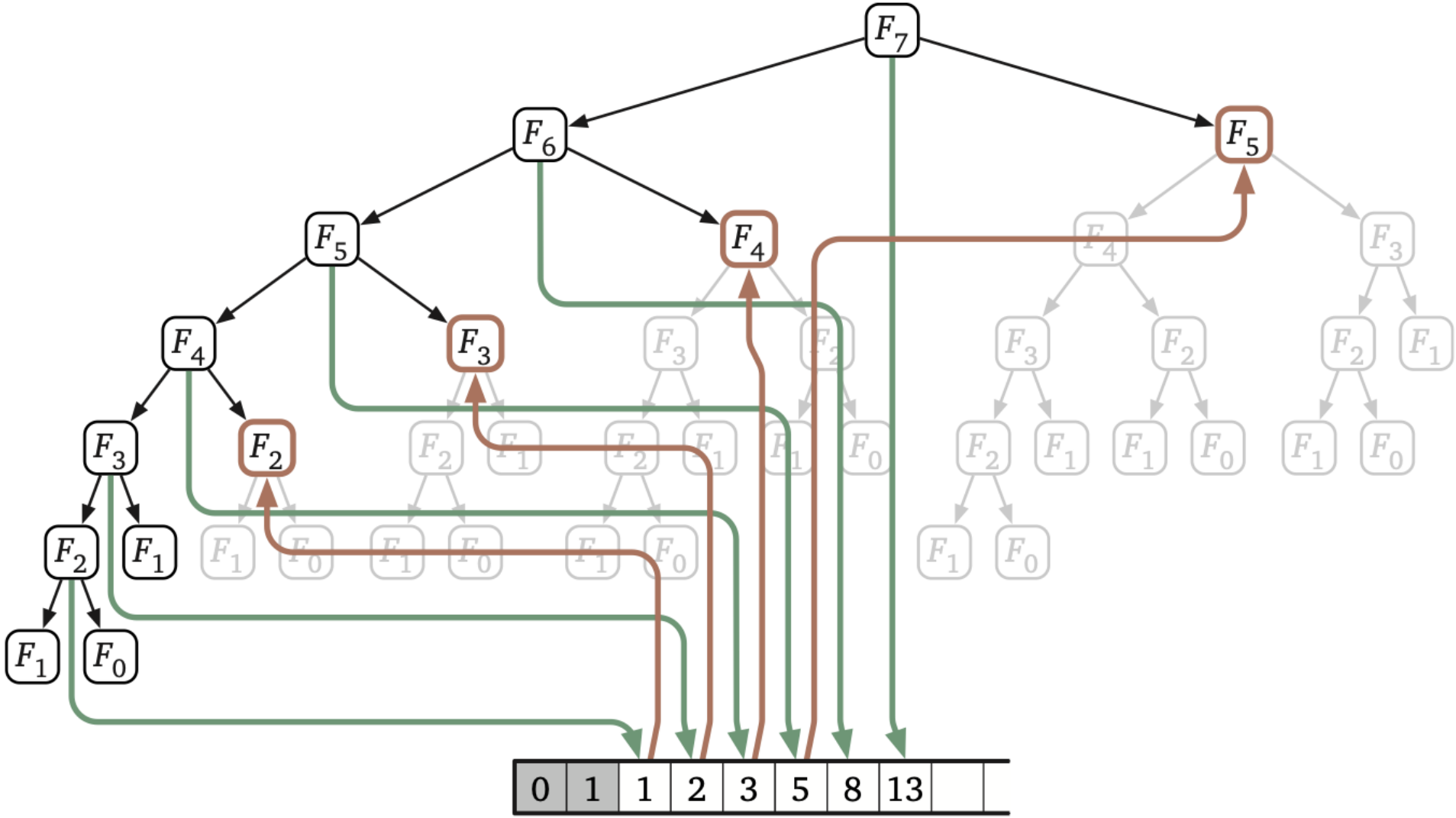


Memo(r)ization: Remember Everything

- เทคนิคการเพิ่มประสิทธิภาพนี้ ซึ่งปัจจุบันรู้จักกันในชื่อว่า **Memoization** (สะกดแบบไม่มีตัว r) โดยทั่วไปมักยกเครดิตให้กับ **Donald Michie** ในปี 1967 แต่ในความเป็นจริง เทคนิคที่มีแนวคิดเดียวกันนี้ได้ถูกเสนอไว้แล้วโดย **Arthur Samuel** ในปี 1959

```
MemFibo(n):
  if n = 0
    return 0
  else if n = 1
    return 1
  else
    if F[n] is undefined
      F[n] <- MemFibo(n-1) + MemFibo(n-2)
    return F[n]
```

Recursion Tree trimmed by Memoization



Our First Dynamic Programming

- เราสังเกตเห็นวิธีการเติมตาราง F[ ] จากภาพก่อนหน้า เราสามารถแทน Memoization ด้วยกันการเติมแบบจงใจตามลำดับที่ควรจะเป็น

```
IterFibo(n):
  F[0] <- 0
  F[1] <- 1
  for i <- 2 to n
    F[i] <- F[i-1] + F[i-2]
  return F[n]
```

- สังเกตได้ชัดเจนว่าอัลกอริทึมนี้เร็ว O(n)

Dynamic Programming

- Richard Bellman เป็นคนที่ทำให้ Dynamic Programming เป็นที่นิยม
- เหตุผลที่เลือกคำนี้ เพราะหัวหน้าของเขาไม่ชอบคณิตศาสตร์ เขาเลือกคำว่า Programming แทน ในสมัยนั้นยังไม่มีคอมพิวเตอร์ ดังนั้นคำว่า Programming มาจากรากศัพท์คำว่า Program ที่หมายถึงการวางแผนเช่น TV Program
- กองทัพอากาศสหรัฐสนับสนุน Bellman และคณะให้พัฒนาวิธีจัดตารางฝึกอบรม ซึ่งพวกเขาเรียกว่า “programs” โดยคำว่า “dynamic” ใช้สื่อถึงกระบวนการหลายขั้นตอนที่เปลี่ยนแปลงตามเวลาและยังช่วยให้ชื่อฟังดูทันสมัยด้วย

Super Fast Fibonacci

- ก่อนหน้านี้เราเขียนวิธีการหาลำดับฟิโบนักชีโดยการใช้ Matrix แต่จริงๆ แล้วมีอีกวิธีหนึ่งที่เร็วเท่าๆ กัน
- เราใช้คุณสมบัติ

  $F_{2n-1} = (F_{n-1})^2 + (F_n)^2$

  $F_{2n} = F_n(2F_{n-1} + F_n)$

  คิดโดย Edouard Lucas ในปี 1898

ลองทำปัญหา C2Q005 อีกรอบ!!!

```
FastRecFibo(n): {F_{n-1}, F_n}
  if n = 1
    return 0, 1
  m <- floor(n/2)
  hprv, hcur <- FastRecFibo(m)
  prev <- hprv^2 + hcur^2
  curr <- hcur * (2 * hprv + hcur)
  next <- prev + curr
  if n is even
    return prev, curr
  else
    return curr, next
```

Dynamic programming ไม่ใช่การเติมค่าในตาราง แต่คือการค้นหา Recursion ที่ฉลาด

“Dynamic programming is not about filling in tables.

It’s about smart recursion!” by Jeff Erickson

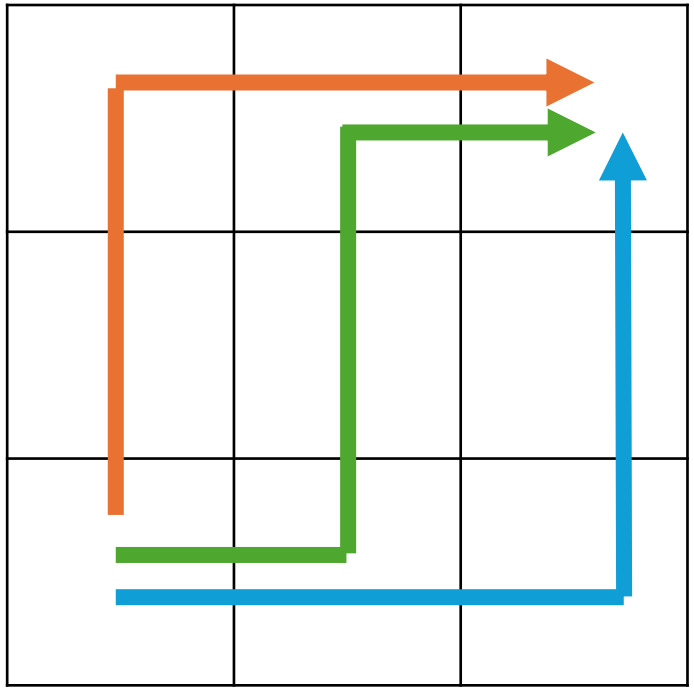
How to Dynamic Programming

Dynamic Programming แบ่งออกเป็นสองขั้นตอนใหญ่ๆ คือ

1) เขียนปัญหาให้อยู่ในรูปแบบ Recursion
   - เขียนให้เคลียร์ว่าคุณกำลังแก้ปัญหาอะไรอยู่ ไม่ใช่แก้อย่างไร
   - เขียนสมการเวียนเกิดที่เอาคำตอบจากปัญหาเดิมที่เล็กลงมารวมกัน
2) เขียนอัลกอริทึมของ Recursion ที่คุณคิดแบบ Bottom Up
   - มีวิธีการเรียก Recursion ได้กี่แบบ
   - เลือกโครงสร้างข้อมูลสำหรับการ Memoization
   - วาดโครงสร้างข้อมูลนั้น เลือกข้อมูลมาหนึ่งจุด ระบุให้ได้ว่าต้องรู้ข้อมูลจุดไหนก่อนบ้าง ถึงจะสามารถคำนวณคำตอบของจุดนี้ได้

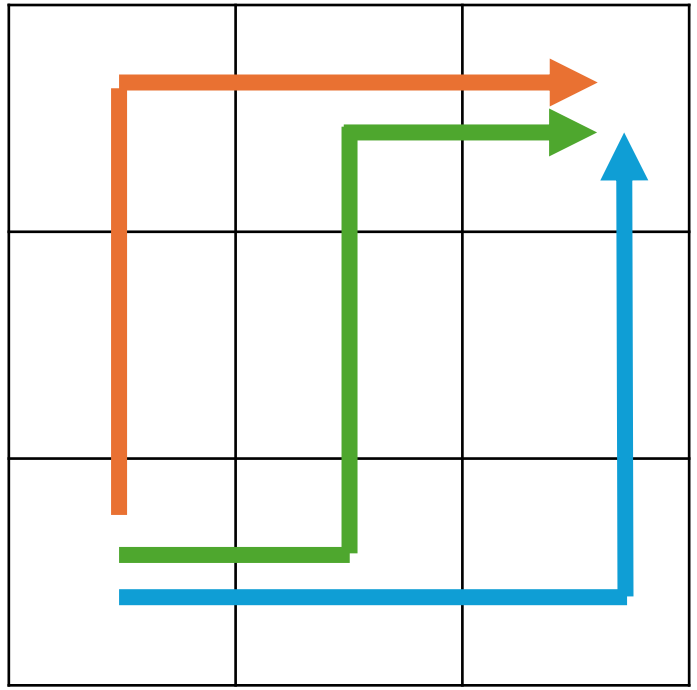
Grid Path

- มาลองนับเลขกันดีกว่า
- เรามีแผนที่ 2D เริ่มเดินจากล่างซ้าย (0,0) ไปบนขวา (W,H) เดินได้แค่ขึ้นหรือขวา มีวิธีการเดินทั้งหมดกี่แบบ



Grid Path – Problem Specification

- โจทย์เดิมคือต้องการหาวิธีการเดินทั้งหมด ซึ่งเราอาจจะยังมองไม่เห็นภาพว่าจะแก้ปัญหานี้อย่าง เพราะฉะนั้นเราจะเขียนโจทย์ใหม่เป็นต้องการหาวิธีการเดินจากจุดเริ่มต้นไปยังจุดใดจุดหนึ่งของแผนที่
- GP(x,y) = จำนวนการเดินจาก (0,0) มายัง (x,y)
- สังเกตว่าปัญหาเราไม่เปลี่ยน แต่ปัญหาอาจเล็กลงได้ กล่าวคือ (x,y) สามารถเป็นจุดก่อนจะถึงปลายทางได้



Grid Path – Recursive Formula

- อันดับต่อไปคือเราต้องการความสัมพันธ์ระหว่าง GP(x,y) กับ G(i,j) เมื่อ i <= x และ j <= y
- สังเกตว่าจะมีเส้นทางอยู่สองประเภทที่จะไปถึง (x,y) คือ เส้นทางที่ผ่าน (x-1,y) และเส้นทางที่ผ่าน (x,y-1) อีกทั้งเส้นทางทั้งสองประเภทไม่ซ้ำกัน เพราะฉะนั้นเราสามารถสรุปได้ว่า GP(x,y) = GP(x-1,y) + GP(x,y-1)

	(x-1,y)	**(x,y)**
		(x,y-1)

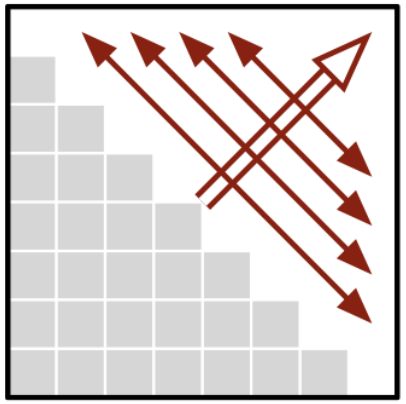
Grid Path – Data Structure

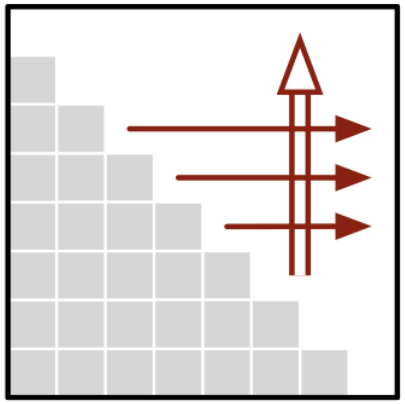
- เราได้ Recursive Formula เป็น GP(x,y) = GP(x-1,y) + GP(x,y-1)
- อันดับต่อไปคือการเลือกโครงสร้างข้อมูลที่จะเก็บค่าทั้งหมด
- ปกติแล้วเราจะดูที่ Parameter ของสมการเรา ในที่นี้เราจะได้ว่า Parameter คือ x และ y ดังนั้นเราจึงเลือกใช้ Array 2 มิติที่มีขนาด W x H
- สังเกตว่าขนาดเท่ากับแผนที่ในโจทย์เลย อันนี้เป็นเรื่องบังเอิญ แต่หลายปัญหาจะเป็นในลักษณะนี้

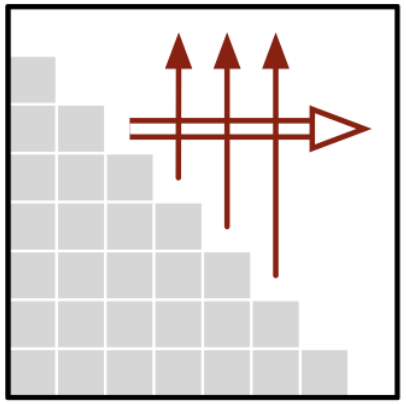
H\W	0	1	2
2			
1			
0			

Grid Path – Evaluation Order

- ลำดับต่อมาคือการลำดับการเติมค่าลงในตาราง เราสังเกตจากสมการ GP(x,y) = GP(x-1,y) + GP(x,y-1) แล้วพบว่าถ้าจะคำนวณค่าที่ตำแหน่ง (x,y) ได้ เราต้องรู้ค่าที่ตำแหน่ง (x-1,y) และ (x,y-1) ก่อน
- เพราะฉะนั้นเราควรเติมค่าจากล่างขึ้นบนและจากซ้ายไปขวา







H\W	0	1	2
2			
1			
0			

Grid Path – Base Case

- สำหรับ Base Case ของปัญหานี้คือช่องล่างซ้ายสุด เพราะไม่มีช่องที่อยู่ทางซ้ายหรืออยู่ด้านล่างของมัน
- GP(0,0) = 1

H\W	0	1	2
2			
1			
0	1		

Grid Path – The Algorithm

- สุดท้ายแล้วเราก็จะได้อัลกอริทึมของปัญหานี้ โดยคำตอบสุดท้ายอยู่ที่ช่อง (W,H)

```
CountGridPath(W,H):
  G[][] <- 0 // initialize all entry to 0
  for x <- 0 to W
    for y <- 0 to H
      if x = 0 and y = 0
        GP[x][y] <- 1 // base case
      if x > 0
        GP[x][y] <- G[x-1][y]
      if y > 0
        GP[x][y] <- G[x][y] + G[x][y-1]
  return GP[W][H]
```

Binary String

- ปัญหาต่อมาคือการนับจำนวนข้อความขนาด n ที่ประกอบไปด้วยตัวเลข 0 หรือ 1 (เรียกว่า Binary String) โดยที่จะต้องไม่มีเลข 1 อยู่ติดกัน
- เช่น ถ้า n = 4 เราจะได้ Binary String เป็น 0000, 0001, 0010, 0100, 0101, 1000, 1001 และ 1010 จำนวนทั้งสิ้น 8 แบบ

- ถ้าไม่มีเงื่อนไขว่าเลข 1 ต้องไม่อยู่ติดกัน จำนวนข้อความจะเป็นเท่าไหร่?

Binary String – Spec and Recurse

- กำหนดให้ BS(k) คือจำนวน Binary String ขนาด k ที่ไม่มี 1 ติดกัน
- Binary String ขนาด k มีด้วยกันสองแบบคือ
  - เริ่มต้นด้วยเลข 0 ในกรณีนี้เลขถัดไปเป็นเลขอะไรก็ได้ เพราะฉะนั้นจำนวน Binary String ขนาด k ที่เริ่มต้นด้วยเลข 0 มีจำนวนทั้งสิ้น BS(k-1)
  - เริ่มต้นด้วยเลข 1 ในกรณีนี้เลขถัดไปเป็นได้แค่เลข 0 แต่เลขถัดจากเลข 0 นั้นเป็นเลขอะไรก็ได้ เพราะฉะนั้นจำนวน Binary String ในกรณีนี้คิดเป็น BS(k-2)
- เราสรุปได้ว่า BS(k) = BS(k-1) + BS(k-2) (คุ้นๆ ไหม)

Binary String – DS / Order / Base

- BS(k) มีวิธีการเรียก n แบบ เราจึงเก็บค่าใน Array 1 มิติขนาด n
- การที่เราจะคำนวณ BS(k) ได้เราต้องรู้ค่าของตัวซ้ายมือก่อน ดังนั้นเราจึงเติมค่าใน Array จากซ้ายไปขวา
- Base Case มีสองค่าคือ k = 1 และ k = 2 เพราะเป็นสองช่องที่ไม่มีทั้งสองตัวก่อนหน้า สำหรับ k = 2 มี 3 แบบคือ 00, 10, 01

k	**1**	**2**	**3**	**4**	**5**	**6**
BS(k)	2	3				

Try it out

- ให้เวลา 10 นาทีทำข้อ C2P52 **Urinal usage II**
- มีโถปัสสาวะ U โถ มีคนใช้งาน N คน ต้องการนับวิธีการใช้งานทั้งหมดที่ไม่มีสองคนใดๆ ยืนติดกัน
- ไอเดียคล้ายกับ Binary String เลยแต่เป็น 2 มิติแทน

Integer Partition

- มีเลขจำนวนเต็มบวกมาให้ ให้นับวิธีการบวกเลขเพื่อให้ได้เลขจำนวนเต็มบวกนั้น
- เช่น n = 4 ก็จะมี 4 แบบคือ

  4

  3 + 1

  2 + 2

  2 + 1 + 1

  1 + 1 + 1 + 1

Integer Partition – Spec and Recurse

- ให้ IP(k) เป็นจำนวนวิธีการบวกให้ได้ k
- ถ้าเราแยกวิธีการบวกโดยดูจากตัวแรก เราจะได้ Recursion
  IP(k) = IP(k-1) + IP(k-2) + ..+ IP(0)
- IP(k-i) หมายถึงตัวเลขตัวแรกคือ i แล้วนับวิธีการบวกให้ได้ k-i

- เจอปัญหาไหม?

Duplication

- IP(3) นับ 3, 2+1, 1+1+1
- IP(2) นับ 2, 1+1
- แต่เมื่อเอามาประกอบร่างกันสำหรับ IP(4) เราจะนับ 1+2+1 จาก IP(3) และ 2+1+1 จาก IP(2) ด้วยกัน ซึ่งจริงๆ แล้วมันคือวิธีเดียวกัน เรานับซ้ำ
- ปัญหาอยู่ที่การเรียงลำดับเลข ถ้าเราไม่อยากนำซ้ำเราควรให้เลขเรียงจากมากไปน้อย เพราะฉะนั้นเราควรกำหนดค่าสูงสุดที่เป็นได้ของวิธีการบวกหลังจากที่เราเลือกตัวเลขตัวแรกแล้ว

Integer Partition – Spec and Recurse v2

- ให้ IP(k,a) เป็นจำนวนวิธีการบวกให้ได้ k โดยตัวเลขในผลบวกมีค่าไม่เกิน a
- ถ้าเราแยกวิธีการบวกโดยดูจากตัวแรก เราจะได้ Recursion

  IP(k,a) = IP(k-a,a) + IP(k-a+1,a-1) + … + P(k-i,i) + … + IP(k-1,1)
- IP(k-i,i) หมายถึงเราเลือกตัวแรกเป็น i ตัวที่เหลือในผลบวกต้องมีค่าไม่เกิน I
- IP(4,4) = IP(0,4) + IP(1,3) + IP(2,2) + IP(3,1)

  IP(0,4) นับ 4 + 0

  IP(1,3) นับ 3 + 1

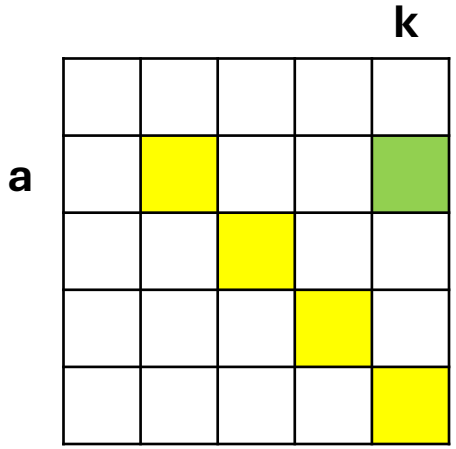
  IP(2,2) นับ 2 + 2 และ 2 + 1 + 1

  IP(3,1) นับ 1 + 1 + 1 + 1

Integer Partition – DS / Order / Base

- IP(k,a) = IP(k-a,a) + IP(k-a+1,a-1) + … + P(k-i,i) + … + IP(k-1,1)
- วิธีการเรียก IP(k,a) เป็นไปได้ตั้งแต่ 1 <= a <= k <= n เพราะฉะนั้นใช้ Array 2 มิติในการเก็บค่า ส่วนลำดับการเติม Array นั้น เราพบว่าเราต้องเติมล่างซ้ายก่อน

```
IP(n,n):
  A[][] <- 0 // initialize all entry to 0
  for k <- 0 to n
    for a <- 0 to k
      A[k][a] <- A[k-1][1]
      for i <- 2 to a
        AP[k][a] <- A[k][a] + A[k-i][i]
  return A[n][n]
```



Not So Fast

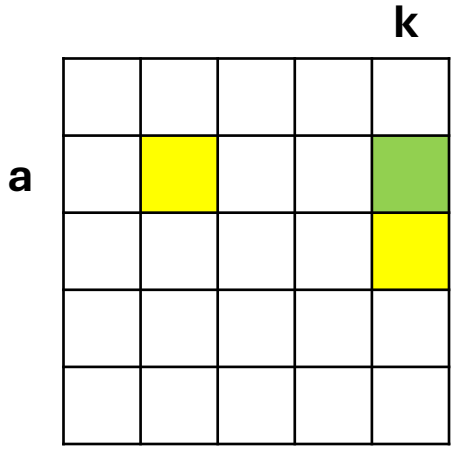
- ถ้าเราวิเคราะห์อัลกอริทึม IP เราจะพบว่าอัลกอริทึมนี้ใช้เวลา $O(n^3)$
- สำหรับโจทย์นี้ถ้าค่า n ไม่เกิน 500 ก็น่าจะไหวอยู่
- ปรากฏว่าโจทย์นี้ให้ค่า n ไว้ถึง 5,000 นั่นแปลว่า $O(n^3)$ ไม่น่าไหว
- แล้วเราควรทำอย่างไรดี?

```
IP(n,n):
  A[][] <- 0 // initialize all entry to 0
  for k <- 0 to n                    // O(n)
    for a <- 0 to k                  // O(n)
      A[k][a] <- A[k-1][1]
      for i <- 2 to a                // O(n)
        AP[k][a] <- A[k][a] + A[k-i][i]
  return A[n][n]
```

Mathematics for the Rescue

- ถ้าเราเก่งคณิตศาสตร์ เราสามารถใช้ทริคนี้ได้
- IP(k,a) = IP(k-a,a) + IP(k-a+1,a-1) + … + IP(k-1,1)
- IP(k,a-1) = IP(k-a+1,a-1) + IP(k-a+2,a-2) + … + IP(k-1,1)
- ส่วนสีเขียวซ้ำกันเพราะฉะนั้นเราจะได้ Recursion เป็น

  IP(k,a) = IP(k-a,a) + IP(k,a-1)
- แค่นี้เราก็ทำให้อัลกอริทึมเราเร็วขึ้นเป็น $O(n^2)$



That is too Magical

- ถ้าเราไม่เก่งเลข เราคงเดาไม่ออกหรอก เพราะฉะนั้นลองมาคิดแบบคอมดูบ้าง
- แบบเดิมเราแบ่งเคสการนับออกเป็น k เคส ตามตัวแรกสุดของผลบวก
- เราต้องคิดใหม่ แบ่งออกเป็นแค่สองเคสคือ
  1. แต่ละตัวในผลบวกใช้ได้แค่ 1, 2, …, a-1
  2. ใช้ a ในผลบวกอย่างน้อย 1 ครั้ง
- สังเกตว่าทั้งสองเคสไม่ซ้ำกันแน่นอน (ดูกันออกไหม)
- เราจะได้ว่า IP(k,a) = IP(k,a-1) + IP(k-a,a) ซึ่งเหมือนกับสไลด์ก่อนหน้าเลย

Try it out

- ให้เวลา 30 นาทีทำข้อ C2P44 **k-way** และ C2P45 **โยนเต๋า**
- ไอเดียคล้ายกับ Integer Partition เลย แก้นิดหน่อยเอง

- ป.ล. ข้อนี้ n ไม่เยอะ เลยไม่ต้อง mod แต่ถ้าเป็นปัญหาแข่งจริง เขาจะให้ mod

References

- Erickson, Jeff. *Algorithms*. 2023.